Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования «Сибирский государственный университет телекоммуникаций и информатики»

(СибГУТИ)

**Кафедра прикладной математики и кибернетики**

# Операционные системы

Лабораторная работа №3

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Галкина М.Ю.

Новосибирск 2024

**Оглавление**

Задание на лабораторную работу ..... 3

Результаты работы программы ……..……. 5

Текст программы ..... 10

# Задание на лабораторную работу

## Нахождение начального опорного плана транспортной задачи

## Нахождение всех базисных решений системы линейных уравнений

**Задание:**

Написать программу, находящую начальный опорный план транспортной задачи одним из указанных методов (номер метода находится как n mod 3, где n – номер бригады).

0. Метод северо-западного угла.
1. Метод минимальной стоимости.
2. Метод Фогеля

**Входные данные:**

На вход программе подаются начальные данные распределительной таблицы: тарифы перевозок, запасы поставщиков и потребности потребителей (данные считываются из файла в виде матрицы размера (m+1)×(n+1)):

| Поставщики | Потребители | | | | Запасы |
|---|---|---|---|---|---|
| | $B_1$ | $B_2$ | … | $B_n$ | |
| $A_1$ | $c_{11}$ | $c_{12}$ | … | $c_{1n}$ | $a_1$ |
| $A_2$ | $c_{21}$ | $c_{22}$ | … | $c_{2n}$ | $a_2$ |
| … | … | … | … | … | … |
| $A_m$ | $c_{m1}$ | $c_{m2}$ | … | $c_{mn}$ | $a_m$ |
| Потребности | $b_1$ | $b_2$ | … | $b_n$ | |

**Требования к программе**:

Программа должна:

− работать как с открытой, так и закрытой моделью транспортной задачи;

− выводить распределительную таблицу после распределения каждой перевозки;

− выводить итоговое распределение перевозок и затраты на перевозки.

Для получения максимальной оценки необходимо предусмотреть программное нахождение вырожденного плана

**Результаты работы программы**

```
nothomepc@DESKTOP-M12GO9R:~/code/3/3$ ./app
Введите название файла:
1a.in

        Исходная таблица:
 Поставщики       B1       B2       B3       B4       B5    Запасы
         A1       13       14       12        9       11        51
         A2       16       17       14       12       15        46
         A3       18       16       15       15       16        99
Потребности       74       23       90       49       34

         Запасы: 196                    Потребности: 270

        Потребностей больше чем запасов, добавлен новый поставщик

        Новая таблица:
 Поставщики       B1       B2       B3       B4       B5    Запасы
         A1       13       14       12        9       11        51
         A2       16       17       14       12       15        46
         A3       18       16       15       15       16        99
         A4        0        0        0        0        0        74
Потребности       74       23       90       49       34

 Поставщики       B1       B2       B3       B4       B5    Запасы  Мин

         A1       13       14       12        9       11        51    2
         A2       16       17       14       12       15        46    2
         A3       18       16       15       15       16        99    0
         A4        0        0        0        0        0        74    0
Потребности       74       23       90       49       34
        Мин        3        2        2        3        4
```

Минимальный элемент: A1B5

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | - | 34 | 17 |
| A2 | - | - | - | - | - | 46 |
| A3 | - | - | - | - | - | 99 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 23 | 90 | 49 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | 13 | 14 | 12 | 9 | - | 17 | 3 |
| A2 | 16 | 17 | 14 | 12 | - | 46 | 2 |
| A3 | 18 | 16 | 15 | 15 | - | 99 | 0 |
| A4 | 0 | 0 | 0 | 0 | - | 74 | 0 |
| Потребности | 74 | 23 | 90 | 49 | 0 | | |
| Мин | 3 | 2 | 2 | 3 | - | | |

Минимальный элемент: A1B4

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | - | - | - | - | 46 |
| A3 | - | - | - | - | - | 99 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 23 | 90 | 32 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 | - |
| A2 | 16 | 17 | 14 | 12 | - | 46 | 2 |
| A3 | 18 | 16 | 15 | 15 | - | 99 | 0 |
| A4 | 0 | 0 | 0 | 0 | - | 74 | 0 |
| Потребности | 74 | 23 | 90 | 32 | 0 | | |
| Мин | 2 | 1 | 1 | 3 | - | | |

Минимальный элемент: A2B4

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | - | - | 32 | - | 14 |
| A3 | - | - | - | - | - | 99 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 23 | 90 | 0 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 | - |
| A2 | 16 | 17 | 14 | - | - | 14 | 2 |
| A3 | 18 | 16 | 15 | - | - | 99 | 1 |
| A4 | 0 | 0 | 0 | - | - | 74 | 0 |
| Потребности | 74 | 23 | 90 | 0 | 0 | | |
| Мин | 2 | 1 | 1 | - | - | | |

Минимальный элемент: A2B3

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | - | 14 | 32 | - | 0 |
| A3 | - | - | - | - | - | 99 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 23 | 76 | 0 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 | - |
| A2 | - | - | - | - | - | 0 | - |
| A3 | 18 | 16 | 15 | - | - | 99 | 1 |
| A4 | 0 | 0 | 0 | - | - | 74 | 0 |
| Потребности | 74 | 23 | 76 | 0 | 0 | | |
| Мин | 0 | 0 | 0 | - | - | | |

Минимальный элемент: A3B3

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | - | 14 | 32 | - | 0 |
| A3 | - | - | 76 | - | - | 23 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 23 | 0 | 0 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 | - |
| A2 | - | - | - | - | - | 0 | - |
| A3 | 18 | 16 | - | - | - | 23 | 2 |
| A4 | 0 | 0 | - | - | - | 74 | 0 |
| Потребности | 74 | 23 | 0 | 0 | 0 | | |
| Мин | 0 | 0 | - | - | - | | |

Минимальный элемент: A3B2

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | 0 | 14 | 32 | - | 0 |
| A3 | - | 23 | 76 | - | - | 0 |
| A4 | - | - | - | - | - | 74 |
| Потребности | 74 | 0 | 0 | 0 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы | Мин |
|---|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 | - |
| A2 | - | - | - | - | - | 0 | - |
| A3 | - | - | - | - | - | 0 | - |
| A4 | 0 | - | - | - | - | 74 | 0 |
| Потребности | 74 | 0 | 0 | 0 | 0 | | |
| Мин | 0 | - | - | - | - | | |

Минимальный элемент: A4B1

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | 17 | 34 | 0 |
| A2 | - | 0 | 14 | 32 | - | 0 |
| A3 | - | 23 | 76 | - | - | 0 |
| A4 | 0 | - | - | - | - | 0 |
| Потребности | 0 | 0 | 0 | 0 | 0 | |

| Поставщики | B1 | B2 | B3 | B4 | B5 | Запасы |
|---|---|---|---|---|---|---|
| A1 | - | - | - | - | - | 0 |
| A2 | - | - | - | - | - | 0 |
| A3 | - | - | - | - | - | 0 |
| A4 | - | - | - | - | - | 0 |
| Потребности | 0 | 0 | 0 | 0 | 0 | |

Z = 2615

## Текст программы

```
#include <fstream>
#include <iomanip>
#include <cstdio>
#include <cstring>
#include <cstdlib>
#include <iostream>
#include <sstream>

using namespace std;

int m, n; // m- columns, n  - lines

// структура минимумов для строчек / столбцов
struct minimum
{
    int min_first = 9999999;
    int min_second = 9999999;
    int i = -1;
    int j = -1;
    int dif = 0;
};

// обнуление массива структур разниц минимумов на новой итерации
void set_struct_to_default(minimum *mas, int size)
{
    for (int i = 0; i < size; i++)
    {
        mas[i].min_first = 9999999;
        mas[i].min_second = 9999999;
        mas[i].i = -1;
        mas[i].j = -1;
        mas[i].dif = 0;
    }
    return;
}

void print(int **mas)
{
    cout << endl
         << " Поставщики";
    for (int i = 1; i < m; i++)
    {
        cout << "\t  B" << i;
    }
    cout << "\tЗапасы" << endl;
    for (int i = 0; i < n; i++)
    {
        if (i == n - 1)
        {
```

```
            cout << "Потребности";
        }
        else
        {
            cout << "\tA" << i + 1;
        }
        for (int j = 0; j < m; j++)
        {
            if (mas[i][j] >= 0)
            {
                printf("\t%3d", mas[i][j]);
            }
            else
            {
                if (mas[i][j] == -10)
                {
                    printf("\t  -");
                }
            }
        }
        cout << endl;
    }
    cout << endl;
}

void print_with_min(int **mas, minimum *min_lines, minimum *min_colls)
{
    cout << endl
         << " Поставщики";
    for (int i = 1; i < m; i++)
    {
        cout << "\t B" << i;
    }
    cout << "\tЗапасы\tМин" << endl;
    for (int i = 0; i < n; i++)
    {
        if (i == n - 1)
        {
            cout << "\nПотребности";
        }
        else
        {
            cout << "\n\t A" << i + 1;
        }
        for (int j = 0; j < m; j++)
        {
            if (mas[i][j] >= 0)
            {
                printf("\t%3d", mas[i][j]);
            }
```

```
            else
            {
                if (mas[i][j] == -10)
                {
                    printf("\t  -");
                }
            }
        }
        if (i < n - 1)
        {
            if (min_lines[i].dif == -10)
            {
                printf("\t  -");
            }
            else
            {
                printf("\t%3d", min_lines[i].dif);
            }
        }
    }
    cout << endl
         << "       Мин";
    for (int i = 0; i < m - 1; i++)
    {
        if (min_colls[i].dif == -10)
        {
            printf("\t  -");
        }
        else
        {
            printf("\t%3d", min_colls[i].dif);
        }
    }
}

int **update_Table(int **arr)
{
    int column_sum = 0;
    for (int i = 0; i < n - 1; i++)
    {
        column_sum += arr[i][m - 1];
    }
    int line_sum = 0;
    for (int i = 0; i < m - 1; i++)
    {
        line_sum += arr[n - 1][i];
    }
    cout << endl
         << "\t Запасы: " << column_sum << "\t\t\t Потребности: " << line_sum <<
endl
```

```
     << endl;

// проверяем сходятся ли запасы с потребностями
if (column_sum == line_sum)
{
    cout << "\tЗапасы удовлетворяют потребностям, таблица не меняется";
    return arr;
}
int **new_arr;
// если потребностей больше - добавляем ещ  одну строку
if (line_sum > column_sum)
{
    n = n + 1;
    new_arr = new int *[n];
    for (int i = 0; i < n - 2; i++)
    {
        new_arr[i] = new int[m];
        for (int j = 0; j < m; j++)
        {
            new_arr[i][j] = arr[i][j];
        }
    }
    // доп строка
    new_arr[n - 2] = new int[m];
    for (int i = 0; i < m - 1; i++)
    {
        new_arr[n - 2][i] = 0;
    }
    new_arr[n - 2][m - 1] = line_sum - column_sum;

    // последняя строка
    new_arr[n - 1] = new int[m];
    for (int i = 0; i < m; i++)
    {
        new_arr[n - 1][i] = arr[n - 2][i];
    }
    cout << "\tПотребностей больше чем запасов, добавлен новый поставщик\n\n";
    return new_arr;
}
// если запасов больше - добавляем ещ  один столбец
else
{
    m = m + 1;
    new_arr = new int *[n];
    for (int i = 0; i < n; i++)
    {
        new_arr[i] = new int[m];
        for (int j = 0; j < m - 2; j++)
        {
            new_arr[i][j] = arr[i][j];
```

```
            }
        }
        // доп столбец
        for (int i = 0; i < n - 1; i++)
        {
            new_arr[i][m - 2] = 0;
        }
        new_arr[n - 1][m - 2] = column_sum - line_sum;

        // последний столбец
        for (int i = 0; i < n; i++)
        {
            new_arr[i][m - 1] = arr[i][m - 2];
        }
        cout << "\tЗапасов больше чем потребностей, добавлен новый потребитель";
        return new_arr;
    }
}

// считает разности двух минимумов для строчек
void count_lines_min(int **arr, minimum *mas)
{
    for (int i = 0; i < n - 1; i++)
    {
        // находим первый минимум в строке
        for (int j = 0; j < m - 1; j++)
        {
            if ((arr[i][j] > 0) && (arr[i][j] < mas[i].min_first))
            {
                mas[i].min_first = arr[i][j];
                mas[i].i = i;
                mas[i].j = j;
            }
        }
        // находим второй минимум в строке
        for (int j = 0; j < m - 1; j++)
        {
            if ((arr[i][j] > 0) && (arr[i][j] < mas[i].min_second) && (j !=
mas[i].j))
            {
                mas[i].min_second = arr[i][j];
            }
        }
        // находим разницу между двумя миимумами
        mas[i].dif = mas[i].min_second - mas[i].min_first;
        // если строка добавочная
        if (mas[i].min_first == 9999999)
        {
            int counter = 0;
            for (int j = 0; j < m - 1; j++)
```

```
            {
                if (arr[i][j] == 0)
                {
                    mas[i].i = i;
                    mas[i].j = j;
                    break;
                }
                counter++;
            }
            // если спрос удовлетвор н и вся строка уже закрашена
            if (counter == m - 1)
            {
                mas[i].dif = -10;
            }
        }
        // если остался единственный элемент в строке
        else
        {
            if (mas[i].min_second == 9999999)
            {
                mas[i].dif = 0;
            }
        }
    }
    return;
}

// считает разности двух минимумов для столбцов
void count_columns_min(int **arr, minimum *mas)
{
    for (int j = 0; j < m - 1; j++)
    {
        // находим первый минимум в столбце
        for (int i = 0; i < n - 1; i++)
        {
            if ((arr[i][j] > 0) && (arr[i][j] < mas[j].min_first))
            {
                mas[j].min_first = arr[i][j];
                mas[j].i = i;
                mas[j].j = j;
            }
        }
        // находим второй минимум в столбце
        for (int i = 0; i < n - 1; i++)
        {
            if ((arr[i][j] > 0) && (arr[i][j] < mas[j].min_second) && (i !=
mas[j].i))
            {
                mas[j].min_second = arr[i][j];
            }
```

```
        }
        // находим разницу между двумя миимумами
        mas[j].dif = mas[j].min_second - mas[j].min_first;
        // если столбец добавочный
        if (mas[j].min_first == 9999999)
        {
            int counter = 0;
            for (int i = 0; i < n - 1; i++)
            {
                if (arr[i][j] == 0)
                {
                    mas[j].i = i;
                    mas[j].j = j;
                    break;
                }
                counter++;
            }
            // если поставки исчерпаны и весь столбец закрашен
            if (counter == n - 1)
            {
                mas[j].dif = -10;
            }
        }
        // если остался единственный элемент в столбце
        else
        {
            if (mas[j].min_second == 9999999)
            {
                mas[j].dif = 0;
            }
        }
    }
    return;
}

minimum find_max(minimum *mas, int size)
{
    minimum max = mas[0];
    for (int i = 1; i < size; i++)
    {
        if (mas[i].dif >= max.dif)
        {
            max = mas[i];
        }
    }
    return max;
}

int check_end(int **arr)
{
```

```
    int counter = 0;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < m - 1; j++)
        {
            if (arr[i][j] != -10)
            {
                counter++;
            }
        }
    }
    if (counter == 0)
    {
        return 1;
    }
    return 0;
}

int take_StockNeedMin(int a, int b)
{
    if (a >= b)
    {
        return b;
    }
    else
    {
        return a;
    }
}

void zero_line(int **arr, int line)
{
    for (int i = 0; i < m - 1; i++)
    {
        arr[line][i] = -10;
    }
}

void zero_collumn(int **arr, int line)
{
    for (int i = 0; i < n - 1; i++)
    {
        arr[i][line] = -10;
    }
}

int main()
{
    setlocale(LC_ALL, "Russian");
    int **arr, **pre_arr, **extra_arr;
```

```
string filename;

cout << "Введите название файла:" << endl;
cin >> filename;
ifstream input(filename);
if (!input)
{
    cout << "Ошибка открытия файла" << endl;
    return 0;
}

input >> n;
input >> m;
m += 1;
n += 1;

pre_arr = new int *[n];

for (int i = 0; i < n; i++)
{
    pre_arr[i] = new int[m];
    for (int j = 0; j < m; j++)
    {
        if (i == n - 1 && j == m - 1)
        {
            pre_arr[i][j] = -100;
        }
        else
            input >> pre_arr[i][j];
    }
}
cout << "\n\tИсходная таблица:";
print(pre_arr);
arr = update_Table(pre_arr);
cout << "\tНовая таблица:";
print(arr);

extra_arr = new int *[n];
for (int i = 0; i < n; i++)
{
    extra_arr[i] = new int[m];
    for (int j = 0; j < m; j++)
    {
        if (j < m - 1 && i < n - 1)
        {
            extra_arr[i][j] = -10;
        }
        else
        {
            extra_arr[i][j] = arr[i][j];
```

```
            }
        }
    }
    minimum min_lines[n - 1];
    minimum min_colls[m - 1];
    minimum result;
    int Z = 0;
    while (true)
    {
        // зануляем массмвы структуп минимальных элементов
        set_struct_to_default(min_lines, n - 1);
        set_struct_to_default(min_colls, m - 1);
        // считаем разницы минимальных элементов по строкам/столбцам
        count_lines_min(arr, min_lines);
        count_columns_min(arr, min_colls);

        print_with_min(arr, min_lines, min_colls);

        // находим максимальные разницы из двух массивов
        minimum line_result = find_max(min_lines, n - 1);
        minimum coll_result = find_max(min_colls, m - 1);

        // определяем какую строку/столбец рассматриваем в завис-ти от максимальной
разницы
        if (line_result.dif >= coll_result.dif)
        {
            result = line_result;
        }
        else
        {
            result = coll_result;
        }
        cout << endl
             << endl
             << endl
             << "\tМинимальный элемент: A" << result.i + 1 << "B" << result.j + 1;

        // выбираем запасы или потребности
        int Available_capacity = take_StockNeedMin(arr[result.i][m - 1], arr[n -
1][result.j]);

        Z += arr[result.i][result.j] * Available_capacity;

        // вычитаем возможный объем из запасов
        arr[result.i][m - 1] -= Available_capacity;
        extra_arr[result.i][m - 1] -= Available_capacity;
        // если запасы кончились, то закрашиваем строку
        if (arr[result.i][m - 1] == 0)
        {
            zero_line(arr, result.i);
```

```
        }

        // вычитаем возможный объ м из потребностей
        arr[n - 1][result.j] -= Available_capacity;
        extra_arr[n - 1][result.j] -= Available_capacity;
        // если потребности кончились, то закрашиваем столбец
        if (arr[n - 1][result.j] == 0)
        {
            zero_collumn(arr, result.j);
        }

        arr[result.i][result.j] = -10;
        extra_arr[result.i][result.j] = Available_capacity;
        if (check_end(arr) == 1)
        {
            break;
        }
        // ставит дополнительный 0
        if ((arr[result.i][m - 1] == 0) && (arr[n - 1][result.j] == 0))
        {
            if ((result.i - 1 > 0) && (extra_arr[result.i - 1][result.j] == -10))
            {
                extra_arr[result.i - 1][result.j] = 0;
            }
            else
            {
                if ((result.i + 1 > 0) && (extra_arr[result.i + 1][result.j] == -
10))
                {
                    extra_arr[result.i + 1][result.j] = 0;
                }
                else
                {
                    if ((result.j - 1 > 0) && (extra_arr[result.i][result.j - 1] ==
-10))
                    {
                        extra_arr[result.i][result.j - 1] = 0;
                    }
                    else
                    {
                        if ((result.j + 1 > 0) && (extra_arr[result.i][result.j + 1]
== -10))
                        {
                            extra_arr[result.i][result.j + 1] = 0;
                        }
                    }
                }
            }
        }
        cout << endl;
```

```
            print(extra_arr);
        }
        extra_arr[result.i][result.j] = 0;
        cout << endl;
        print(extra_arr);
        print(arr);
        cout << "\nZ = " << Z << "\n";
        return 0;
}
```